Roll No: _____ **Date:** _____

Experiment-4b
CPU Scheduling

Program 1: Round Robin Job Scheduling

Aim: Write a program to demonstrate Round Robin job scheduling.

Description

This program simulates the Round Robin (RR) CPU scheduling algorithm, which is a preemptive scheduling algorithm. It is widely used in time-sharing systems, where each process is assigned a fixed Time Quantum (TQ).

- Processes are scheduled in the order of their arrival.
- Each process is allowed to run for at most TQ units of time.
- If the process finishes within that time, it completes execution.
- If not, it is preempted and put back into the ready queue for another chance later.
- This continues until all processes are completed.

The program:

1. Takes number of processes, arrival times (AT), burst times (BT), and the time quantum (TQ).
2. Maintains a ready queue for processes.
3. Executes them in a Round Robin manner while recording:
    - Completion Time (CT)
    - Turnaround Time ($TAT = CT - AT$)
    - Waiting Time ($WT = TAT - BT$)
4. Prints a Gantt chart of execution order.
5. Calculates average waiting time and average turnaround time.

Algorithm

1. **Input**
   - Number of processes n.
   - For each process: Arrival Time (AT[i]), Burst Time (BT[i]).
   - Time Quantum (TQ).
2. **Initialization**
   - $RT[i] = BT[i]$ (Remaining Time).
   - $time$ = minimum arrival time.
   - Initialize an empty ready queue.
   - Push processes that have arrived at `time` into the queue.
3. **Scheduling Loop (`while remain > 0`)**
   - If the ready queue is empty, increment `time` (CPU idle) and check for new arrivals.
   - Else, dequeue the front process `idx`.
   - Execute it for: $exec = \min(RT[idx], TQ)$ units.
   - Update $time+ = exec$, $RT[idx]- = exec$.
   - During execution, check if any new processes have arrived, add them to the queue.
   - If $RT[idx] > 0$, put it back in the queue.
   - Else (process finished):
     - $CT[idx] = time$.
     - $TAT[idx] = CT[idx] - AT[idx]$.
     - $WT[idx] = TAT[idx] - BT[idx]$.
     - Update average totals.
     - `remain--`.
4. **After all processes finish**
   - Print Gantt Chart.
   - Print a table with AT, BT, CT, WT, TAT.
   - Print average WT and average TAT.

Source Code

```c
#include <stdio.h>

int main() {
    int n, tq;
    int pid[20], at[20], bt[20], rt[20], wt[20], tat[20], ct[20];
```

```c
    int time = 0, remain, i, front = 0, rear = 0, queue[100];
    int inQueue[20] = {0};
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    remain = n;

    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("P%d AT BT: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    // Start from earliest arrival
    int min_at = at[0];
    for (i = 1; i < n; i++) {
        if (at[i] < min_at) min_at = at[i];
    }
    time = min_at;

    // Add first arrivals in correct order
    for (int atime = min_at; atime <= time; atime++) {
        for (i = 0; i < n; i++) {
            if (at[i] == atime && !inQueue[i] && rt[i] > 0) {
                queue[rear++] = i;
                inQueue[i] = 1;
            }
        }
    }

    printf("\nGantt Chart: ");
    while (remain > 0) {
        if (front == rear) {
            time++; // CPU idle
            for (int atime = min_at; atime <= time; atime++) {
                for (i = 0; i < n; i++) {
                    if (at[i] == atime && !inQueue[i] && rt[i] > 0) {
                        queue[rear++] = i;
                        inQueue[i] = 1;
                    }
                }
            }
            continue;
        }

        int idx = queue[front++]; // take first from queue
        int exec = (rt[idx] > tq) ? tq : rt[idx];
        printf("| P%d ", pid[idx]);

        time += exec;
        rt[idx] -= exec;
```

```
63
64          // Add newly arrived processes in correct order
65          for (int atime = min_at; atime <= time; atime++) {
66              for (i = 0; i < n; i++) {
67                  if (at[i] == atime && !inQueue[i] && rt[i] > 0) {
68                      queue[rear++] = i;
69                      inQueue[i] = 1;
70                  }
71              }
72          }
73
74          if (rt[idx] > 0) {
75              queue[rear++] = idx; // put back in queue
76          } else {
77              ct[idx] = time;
78              tat[idx] = ct[idx] - at[idx];
79              wt[idx] = tat[idx] - bt[idx];
80              avg_wt += wt[idx];
81              avg_tat += tat[idx];
82              remain--;
83          }
84      }
85
86      printf("|\n\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
87      for (i = 0; i < n; i++) {
88          printf("P%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i
    ], wt[i], tat[i]);
89      }
90
91      printf("\nAverage Waiting Time: %.2f", avg_wt / n);
92      printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
93
94      return 0;
95  }
```

Output

```
student@cselab:~$ gcc roundrobin.c
student@cselab:~$ ./a.out
Enter number of processes: 6
Enter Arrival Time and Burst Time for each process:
P1 AT BT: 5 5
P2 AT BT: 4 6
P3 AT BT: 3 7
P4 AT BT: 1 9
P5 AT BT: 2 2
P6 AT BT: 6 3
Enter Time Quantum: 3

Gantt Chart: | P4 | P5 | P3 | P2 | P4 | P1 | P6 | P3 | P2 | P4 | P1 | P3 |

Process AT      BT      CT      WT      TAT
P1      5       5       32      27      27
```

```
P2      4       6       27      17      23
P3      3       7       33      23      30
P4      1       9       30      20      29
P5      2       2       6       2       4
P6      6       3       21      12      15

Average Waiting Time: 16.83
Average Turnaround Time: 21.33

student@cselab:~$ ./a.out
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 AT BT: 0 6
P2 AT BT: 1 2
P3 AT BT: 2 4
Enter Time Quantum: 2

Gantt Chart: | P1 | P2 | P3 | P1 | P3 | P1 |

Process AT      BT      CT      WT      TAT
P1      0       6       12      6       12
P2      1       2       4       1       3
P3      2       4       10      4       8

Average Waiting Time: 3.67
Average Turnaround Time: 7.67
```

Program 2: Priority Scheduling (Preemptive)

Aim: Write a program to demonstrate Priority Scheduling (Preemptive).

Description

This program simulates the Preemptive Priority CPU Scheduling algorithm. Each process has:

- Arrival Time (AT)
- Burst Time (BT)
- Priority (PR) (lower value = higher priority)

At every time unit:

- The process with the highest priority (lowest PR value) among those that have arrived is chosen.
- The process executes for 1 unit of CPU time.
- If a new process arrives with higher priority, it preempts the currently running one.
- This continues until all processes finish.

The program calculates: Completion Time (CT), Turnaround Time (TAT = CT - AT), Waiting Time (WT = TAT - BT), Average WT, and Average TAT.

Algorithm

1. **Input**
   - Number of processes n.
   - For each process: Arrival Time (AT[i]), Burst Time (BT[i]), Priority (PR[i]).
   - Initialize $RT[i] = BT[i]$ (Remaining Time).
2. **Initialization**
   - Set `time = minimum arrival time`.
   - `remain = n` (number of processes left to finish).
3. **Scheduling Loop (`while remain > 0`)**
   - At each time unit, find the process `highest_pr` such that:
     - It has arrived ($AT[i] \leq time$)
     - It still has remaining time ($RT[i] > 0$)
     - It has the highest priority (lowest PR value) among all eligible processes.
   - If no process is ready → increment `time` (CPU idle).
   - Else run that process for 1 unit of time:

- `RT[highest_pr]--`
- `time++`

- If after execution, $RT[highest_pr] == 0$:
  - Mark completion:
  - $CT[i] = time$
  - $TAT[i] = CT[i] - AT[i]$
  - $WT[i] = TAT[i] - BT[i]$
  - Add to totals for averages.
  - Decrement `remain`.

4. **After all processes finish**
   - Print table of AT, BT, PR, CT, WT, TAT.
   - Print average WT and average TAT.

Source Code

```c
#include <stdio.h>
#include <limits.h>

int main() {
    int n;
    int pid[20], at[20], bt[20], pr[20];
    int rt[20], ct[20], wt[20], tat[20];
    int time = 0, remain, i, highest_pr;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    remain = n;

    printf("Enter Arrival Time, Burst Time, and Priority (lower number = higher priority):\n");
    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("P%d AT BT PR: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i]; // remaining time
    }

    // Start from the earliest arrival
    int min_at = at[0];
    for (i = 1; i < n; i++) {
        if (at[i] < min_at) {
            min_at = at[i];
        }
    }
    time = min_at;

    while (remain > 0) {
        highest_pr = -1;
        int min_priority = INT_MAX;
```

```
35
36          // Select process with highest priority among arrived ones
37          for (i = 0; i < n; i++) {
38              if (at[i] <= time && rt[i] > 0 && pr[i] < min_priority) {
39                  min_priority = pr[i];
40                  highest_pr = i;
41              }
42          }
43
44          if (highest_pr == -1) {
45              time++; // No process arrived yet
46              continue;
47          }
48
49          rt[highest_pr]--; // run for 1 unit
50          time++;
51
52          // If process finished
53          if (rt[highest_pr] == 0) {
54              ct[highest_pr] = time;
55              tat[highest_pr] = ct[highest_pr] - at[highest_pr];
56              wt[highest_pr] = tat[highest_pr] - bt[highest_pr];
57
58              avg_wt += wt[highest_pr];
59              avg_tat += tat[highest_pr];
60              remain--;
61          }
62      }
63
64      printf("\nProcess\tAT\tBT\tPR\tCT\tWT\tTAT\n");
65      for (i = 0; i < n; i++) {
66          printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
67                 pid[i], at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
68      }
69
70      printf("\nAverage Waiting Time: %.2f", avg_wt / n);
71      printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
72
73      return 0;
74  }
```

Output

```
student@cselab:~$ gcc priorityp.c
student@cselab:~$ ./a.out
Enter number of processes: 4
Enter Arrival Time, Burst Time, and Priority (lower number = higher priority):
P1 AT BT PR: 0 8 2
P2 AT BT PR: 2 5 1
P3 AT BT PR: 3 2 0
P4 AT BT PR: 5 20 2

Process AT      BT      PR      CT      WT      TAT
P1      0       8       2       17      9       17
```

```
P2      2       5       1       9       2       7
P3      3       2       0       5       0       2
P4      5       20      2       37      12      32

Average Waiting Time: 5.75
Average Turnaround Time: 14.50
```

Program 3: Priority Scheduling (Non-Preemptive)

Aim: Write a program to demonstrate Priority Scheduling (Non-Preemptive).

Description

This program simulates Non-Preemptive Priority Scheduling, where:

- Each process has an Arrival Time (AT), Burst Time (BT), and Priority (PR) (lower number = higher priority).
- The CPU selects the process with the highest priority among all those that have already arrived and are not yet completed.
- Once a process starts execution, it runs until completion (non-preemptive → cannot be interrupted, even if a higher-priority process arrives in between).
- Ties are broken by:
  1. Earlier AT gets preference.
  2. If still tied, by Process ID (smaller PID first).

Algorithm

1. **Input**
   - Read number of processes n.
   - For each process: Arrival Time (AT[i]), Burst Time (BT[i]), Priority (PR[i]).
2. **Initialization**
   - `time = minimum AT` (start when the first process arrives).
   - `done[i] = 0` for all processes (not completed).
   - `completed = 0`.
3. **Scheduling Loop (`while completed < n`)**
   - From all processes that have arrived ($AT[i] \leq time$) and are not finished ($done[i] == 0$):
   - Select the process with:
     - Highest priority (lowest PR value).
     - If tied, the earlier AT.
     - If tie still, lower PID.
   - If no process is available → increment `time` (CPU idle).
   - Else, run the chosen process until completion:
     - $time+ = BT[idx]$
     - $CT[idx] = time$

- $TAT[idx] = CT[idx] - AT[idx]$
- $WT[idx] = TAT[idx] - BT[idx]$
- Mark it completed: $done[idx] = 1$
- `completed++`

4. **After all processes finish**
   - Print process table (AT, BT, PR, CT, WT, TAT).
   - Print Average Waiting Time and Average Turnaround Time.

Source Code

```c
#include <stdio.h>
#include <limits.h>

int main() {
    int n;
    int pid[50], at[50], bt[50], pr[50];
    int ct[50], wt[50], tat[50], done[50] = {0};
    int time = 0, completed = 0;
    float avg_wt = 0.0f, avg_tat = 0.0f;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival Time, Burst Time, and Priority (lower = higher priority):\n");
    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("P%d AT BT PR: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }

    // Start time at earliest arrival (handles idle CPU before first process)
    int min_at = at[0];
    for (int i = 1; i < n; i++) {
        if (at[i] < min_at) {
            min_at = at[i];
        }
    }
    time = min_at;

    while (completed < n) {
        int idx = -1;
        int best_pr = INT_MAX;
        int best_at = INT_MAX;
        int best_pid = INT_MAX;

        // Pick the highest-priority process among those that have arrived and are not done.
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time) {
                if (pr[i] < best_pr ||
```

```
                    (pr[i] == best_pr && at[i] < best_at) ||
                    (pr[i] == best_pr && at[i] == best_at && pid[i] <
    best_pid)) {
                     best_pr = pr[i];
                     best_at = at[i];
                     best_pid = pid[i];
                     idx = i;
                }
            }
        }

        if (idx == -1) {
            // No process has arrived yet or all arrived ones are
    finished -> CPU idle
            time++;
            continue;
        }

        // Non-preemptive: run the selected process to completion
        time += bt[idx];
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        avg_wt += wt[idx];
        avg_tat += tat[idx];
        done[idx] = 1;
        completed++;
    }

    printf("\nProcess\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);

    return 0;
}
```

Output

```
student@cselab:~$ gcc prioritynp.c
student@cselab:~$ ./a.out
Enter number of processes: 4
Enter Arrival Time, Burst Time, and Priority (lower = higher priority):
P1 AT BT PR: 0 5 2
P2 AT BT PR: 1 3 1
P3 AT BT PR: 2 8 4
P4 AT BT PR: 3 6 3

Process AT      BT      PR      CT      WT      TAT
```

P1	0	5	2	5	0	5
P2	1	3	1	8	4	7
P3	2	8	4	22	12	20
P4	3	6	3	14	5	11

```
Average Waiting Time: 5.25
Average Turnaround Time: 10.75
```